ZRust: A Rust Reimplementation of DrugZ for High-Performance CRISPR Chemogenetic Screen Analysis

Yoon Sing Yap
Independent Researcher
yoonatworkz@gmail.com

Abstract

ZRust is an independent Rust reimplementation of DrugZ that reduces the runtime of pooled CRISPR screen analysis. We used the SUM149PT breast cancer olaparib screen, HeLa olaparib screen, and hTERT-RPE1 epithelial-cell olaparib screen to compare the two programs. ZRust closely reproduced the gene-level scores from pinned Python DrugZ across three published olaparib screens. Pearson correlation and Spearman rank correlation were both 1.000000 in the SUM149PT, HeLa, and hTERT-RPE1 comparisons. In the HeLa olaparib screen, two genes with the same score appeared in the opposite order, but this did not change their statistical results or which genes were identified as significant. On the tested macOS ARM64 workstation, ZRust was 10.8 to 12.3 times faster than pinned Python DrugZ across three published-screen workloads. This comparison used serial output formatting: one worker prepared result rows for writing. Eight workers further reduced runtime for large paired workloads by preparing completed result rows for writing. The scientific calculations remain single-threaded. The largest tested input contained one million guides. These findings describe software agreement and performance on the tested workstation and workloads. They do not establish new biological findings or validation on other platforms.

Background

DrugZ identifies genes that change how cells respond to a treatment in pooled CRISPR screens. Repeated analyses can become slow when a study includes many conditions or large count matrices. DrugZ calculates changes in guide abundance, estimates variation at similar abundance levels, and combines guide scores into gene-level results [1]. Users need faster analyses that preserve these statistical results.

We developed ZRust as an independent Rust reimplementation of DrugZ. ZRust follows the same core analysis while adding clearer input checks, safer file writing, and faster execution. We compared it with a fixed version of the original Python DrugZ software using three published pooled CRISPR olaparib chemogenetic screens and synthetic datasets. The SUM149PT breast cancer olaparib screen, HeLa olaparib screen, and hTERT-RPE1 epithelial-cell olaparib screen provided the biological backgrounds for comparing the two programs. We call this fixed reference pinned Python DrugZ. The study evaluates software behavior and runtime. It does not reinterpret the biological findings of the screens.

Results

How ZRust works

ZRust reads a guide-count matrix and the selected control and treatment samples. It returns gene-level results and can also return guide-level results (Figure 1). It normalizes counts, calculates fold changes, estimates local variation, and calculates guide Z-scores. It then combines guide scores by gene. Separate probabilities and false discovery rate (FDR) adjustments describe sensitization and resistance. FDR adjustment accounts for testing many genes. Paired analysis matches samples by their position in the supplied lists. Unpaired analysis compares the mean of each group and supports unequal group sizes.

ZRust checks that numerical values are finite, meaning neither infinite nor undefined. It first writes each result to a temporary file. It moves the file into its final location only after the write is complete. This protection applies to each output file separately. For large result files, ZRust divides completed rows among several CPU workers. It then joins the rows in their original order. This shortens file-preparation time without changing the scientific calculations. The scientific calculations remain single-threaded. The CPU workers only prepare completed result rows for writing. Eight workers are the tested command-line default. Small outputs use one worker.

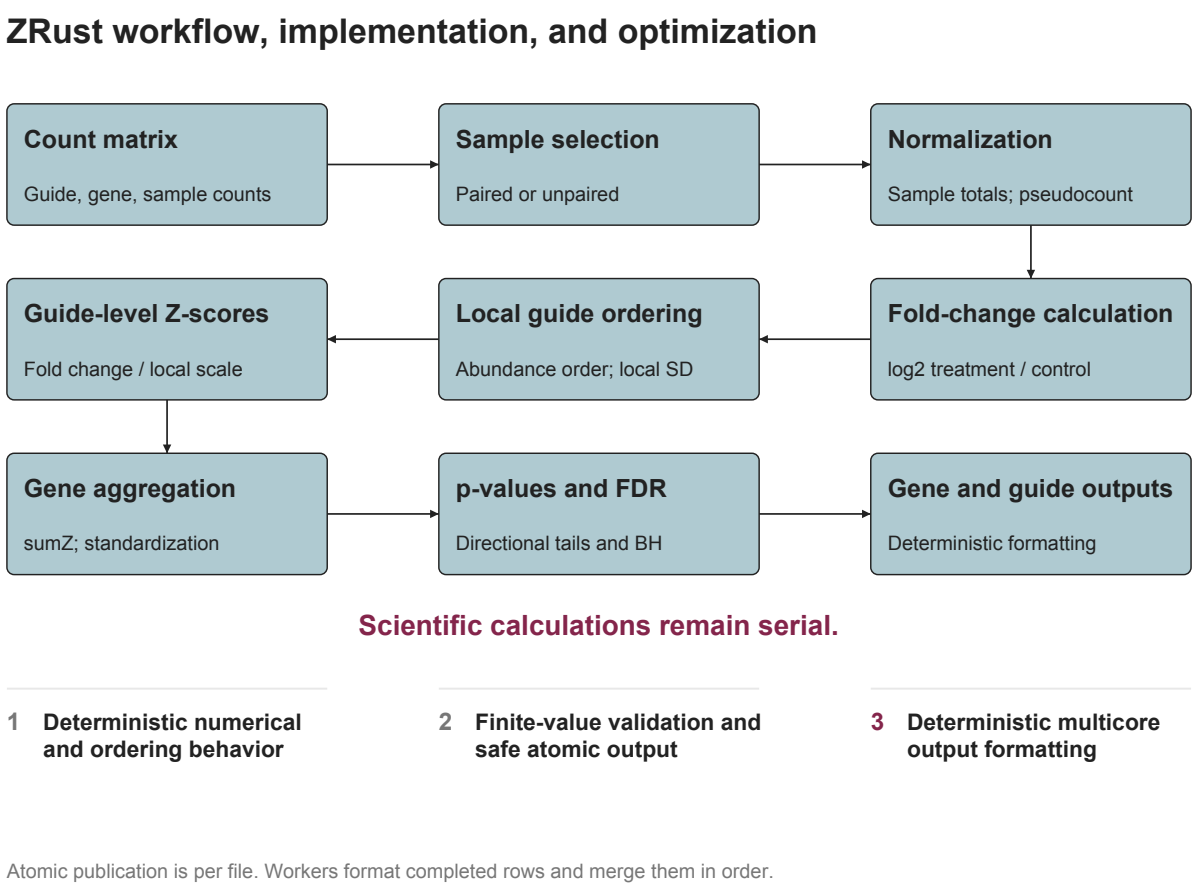


Figure 1: ZRust workflow, implementation, and optimization. ZRust selects samples, normalizes counts, and calculates fold changes. It orders guides by control abundance to estimate local variation, then calculates guide Z-scores and gene-level results. Directional p-values and FDR adjustments describe sensitization and resistance. The callouts summarize repeatable calculations and output order, checks for finite values, safe writing of each file, and preparation of completed rows by CPU workers. Here, deterministic means that repeated runs and different worker settings retain the same output bytes. The scientific calculations remain single-threaded. Workers join the completed rows in their original order.

Comparison with Python DrugZ

ZRust closely reproduced the results from pinned Python DrugZ across the SUM149PT olaparib screen, HeLa olaparib screen, and hTERT-RPE1 olaparib screen (Figure 2a). Gene-level scores had a Pearson correlation of 1.000000 in all three screens. Their rank order also had a Spearman correlation of 1.000000. The SUM149PT olaparib screen and hTERT-RPE1 olaparib screen matched across all tested result fields.

The HeLa olaparib screen had one minor difference. OR4D10 and RPL7 received the same score but appeared in the opposite order. This switched their adjacent rank positions. It did not change their statistical results, direction of effect, or the set of significant genes. We therefore report the HeLa comparison with this minor ordering exception.

Correlation alone does not test every part of the output. We therefore also compared gene membership, individual result values, direction of effect, statistical significance, output order, and gene ranks. We used the same numerical comparison criteria for all three screens. The HeLa olaparib screen exception did not relax these criteria. This study compares ZRust with the pinned Python implementation. It does not attempt to recreate result tables reported in the original publication.

Runtime compared with Python DrugZ

ZRust was faster than pinned Python DrugZ for all three published-screen workloads on the tested workstation (Figure 2b). Median runtimes for ZRust and Python DrugZ were 0.36 and 3.88 seconds for the SUM149PT olaparib screen, 0.31 and 3.81 seconds for the HeLa olaparib screen, and 0.27 and 3.13 seconds for the hTERT-RPE1 olaparib screen. The corresponding speedups were 10.78, 12.29, and 11.59 times. ZRust used serial output formatting in this comparison. Each measurement covered the complete command-line run, including reading input, calculations, and writing completed result files.

Synthetic inputs tested larger matrices. At one million guides, median paired runtime was 2.78 seconds for ZRust and 33.93 seconds for pinned Python DrugZ. Median unpaired runtime was 1.60 and 22.69 seconds, respectively. These ZRust runs also used serial output formatting. Paired runs wrote gene and guide results. Unpaired runs wrote gene results only. These measurements apply to the tested workloads and workstation.

Benefit of using multiple formatting workers

Using eight workers to prepare result rows reduced total runtime for large paired workloads (Figure 2c). At 500,000 guides, median runtime fell from 1.300278 seconds with one worker to 0.900773 seconds with eight workers. At one million guides, it fell from 2.693772 to 1.884198 seconds. These were 1.444- and 1.430-fold improvements. Figure 2c shows the three screens and three paired synthetic sizes. The scientific calculations remained single-threaded.

Eight workers became the default after evaluation across 15 workloads. The benefit of additional workers varied by workload, and one smaller unpaired test did not improve. The Python comparison and the worker comparison used separate configurations and timing methods. Combining their values would not give a directly measured Python-to-eight-worker speedup. Stress tests reached one million guides and 250,000 genes on macOS ARM64. Separate tests checked unpaired analysis with unequal group sizes.

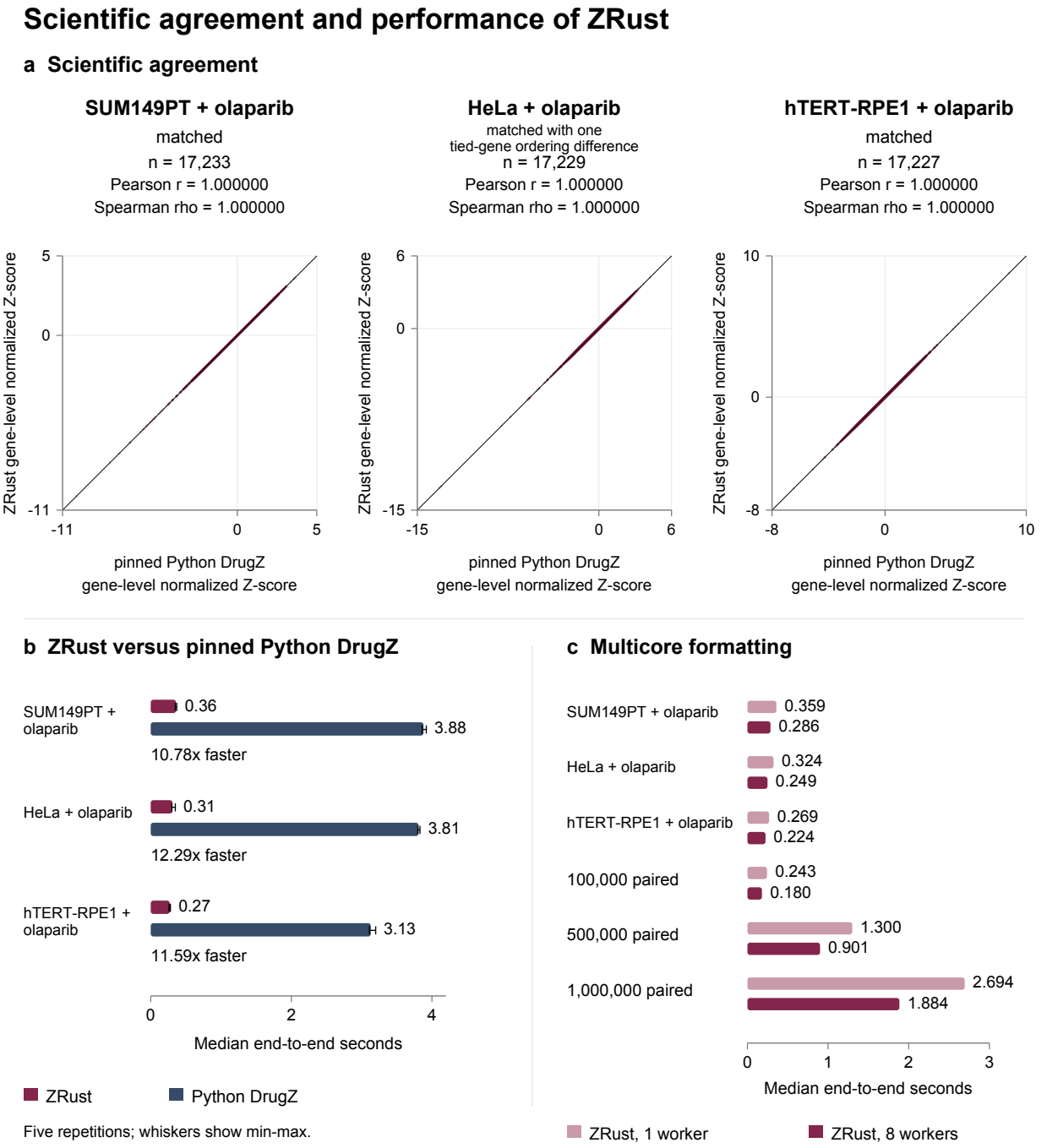


Figure 2. Scientific agreement and performance of ZRust. (a) Gene-level normalized Z-scores from ZRust and pinned Python DrugZ for the SUM149PT olaparib screen, HeLa olaparib screen, and hTERT-RPE1 olaparib screen. Pearson correlation and Spearman rank correlation were both 1.000000 in all three comparisons. The diagonal line marks equal scores. The SUM149PT olaparib screen and hTERT-RPE1 olaparib screen matched under all tested criteria. In the HeLa olaparib screen, OR4D10 and RPL7 received the same score but appeared in the opposite order. This minor ordering difference did not affect their statistical results or the set of significant genes. We report the HeLa olaparib screen with this ordering exception. (b) Median total runtime for ZRust and pinned Python DrugZ. ZRust used serial output formatting for this comparison. (c) Total runtime with one and eight ZRust formatting workers. The size-labeled workloads are synthetic paired guide matrices. The scientific calculations remained single-threaded. Performance results apply only to the tested workloads and macOS ARM64 workstation. The largest tested input contained one million guides.

Discussion and limitations

Across three published olaparib screens, ZRust reproduced gene-level scores and ranks from pinned Python DrugZ with correlations of 1.000000 to six decimal places. Detailed comparisons also showed agreement in gene membership, statistical values, direction of effect, and significant genes. The only observed difference was the order of two tied genes in the HeLa olaparib screen. The SUM149PT olaparib screen and hTERT-RPE1 olaparib screen matched across all tested result fields. Correlation alone does not establish complete field-level agreement. Keeping the tied-gene exception explicit helps distinguish a rank difference from a score difference that could change interpretation.

ZRust also reduced DrugZ analysis time. Preparing completed rows with several workers further shortened large paired runs. These workers do not perform the scientific calculations. This design offers a practical runtime benefit while keeping the calculation order unchanged.

Validation remains limited to finite inputs and source use on the tested macOS ARM64 workstation. One million guides is the largest tested input. Larger matrices have not been validated. Matrix shape, sample count, storage, and operating system may affect runtime and memory use. Extra workers may help less when reading input or performing calculations takes most of the time. Building ZRust on another platform would still require numerical and performance checks.

The published screens tested agreement on existing biological data. Synthetic inputs tested known matrix sizes and failure conditions. Neither type of input provides new evidence about drug mechanisms. Users still need to assess whether DrugZ assumptions fit their study and retain appropriate experimental controls. Future evaluations could cover more platforms and screen designs.

Methods

Calculations and ordering

ZRust removes excluded genes before calculating totals for the selected samples. It normalizes each count as 10^7 times the count divided by the sample total. It then adds the pseudocount and calculates the base-two logarithm of the treatment-to-control ratio. Unpaired analysis first calculates the arithmetic mean of normalized counts within each group. Local variation follows the behavior of pinned Python DrugZ [3]. It uses a mean-centered sample standard deviation, a step-25 window progression, a running maximum, and a final-tail overwrite. These details define the reference software more precisely than the broad method description in the original paper [1].

ZRust reproduces the descending guide order used by the reference through pandas and NumPy, including tied abundance values. This matters because guide order determines membership in the windows used to estimate variation. Paired comparisons use exact integer ordering keys when all original values in the selected control column are integers. Other keys use finite floating-point values. These include raw control means in unpaired analysis. Guide Z is fold change divided by local scale. ZRust sums guide Z-scores in the defined guide and comparison order. It counts nonzero observations, divides the sum by the square root of that count, and standardizes scores across retained genes using population standard deviation. Separate normal-distribution tails and Benjamini-Hochberg adjustments describe sensitization and resistance [2]. Final gene-score ties use ascending UTF-8 identifier order, a fixed order based on the encoded gene names.

Software comparison and stress tests

The author-released count matrices [4] came from the SUM149PT breast cancer olaparib screen, HeLa olaparib screen, and hTERT-RPE1 epithelial-cell olaparib screen. Each contained 91,320 guides. The SUM149PT olaparib screen and HeLa olaparib screen used three paired comparisons. The hTERT-RPE1 olaparib screen used two. Settings were pseudocount five, half-window size 500, minimum observation count one, identifier column zero, and no exclusions. We compared ZRust with the Hart Lab Python DrugZ implementation pinned at commit eb15d34e4dd172965e618d5bb662c053066da799. The environment used Python 3.9.6, NumPy 1.23.5, pandas 1.5.3, and SciPy 1.10.1.

Numerical agreement required an absolute difference no greater than 10^-10 plus 10^-9 times the larger absolute value. Directional p-values or FDR values below 10^-12 also required an absolute difference no greater than 10^-12. Gene membership, order, and ranks had to match exactly, apart from the documented tie in the HeLa olaparib screen. Within each program, OR4D10 and RPL7 had equal scores, directional p-values, and FDR values. Python DrugZ assigned their resistance ranks as 8510 and 8509, respectively. ZRust assigned 8509 and 8510.

Synthetic matrices ranged from 10,000 to 1,000,000 guides. They contained four guides per gene and two control plus two treatment samples. Paired and unpaired stress tests checked numerical agreement, repeated output, handling of nonfinite values, and file-writing behavior. Separate fixtures tested unpaired analysis with unequal group sizes. The million-guide matrix did not use unequal group sizes.

We compared full-precision calculations separately from written result files. Repeated Rust runs had to produce identical output bytes across different worker settings and execution schedules. This is the deterministic output requirement. Comparisons between Python and Rust used the numerical tolerances above.

Figure 2a uses gene identifiers matched between retained full-precision Python DrugZ and ZRust results. The matched sets contained 17,233 genes for the SUM149PT olaparib screen, 17,229 for the HeLa olaparib screen, and 17,227 for the hTERT-RPE1 olaparib screen. No values were missing or nonfinite. Pearson correlation used score values. Spearman correlation used average ranks, with equal scores assigned tied ranks. Every matched gene appears in the plots. We retained the existing matched values and correlations for this editorial revision. We did not recalculate correlations. Complete agreement also requires the individual field comparisons described above.

Implementation and timing

All scientific calculations remain single-threaded, including the ordered sums of floating-point values. One optimization delays creating diagnostic messages for six fold-change checks until a nonfinite value is found. Workers prepare contiguous groups of completed output rows and join them in their original order. If a worker cannot start, the same group is prepared synchronously by the calling thread. The command-line default and maximum are eight workers. We evaluated one, two, four, and eight workers. Gene outputs below 25,000 rows and guide outputs below 50,000 rows use serial formatting. The basic library analysis entry point is also serial.

ZRust supports minimum-observation filtering, quiet operation, and a defined set of columns for unpaired guide output. Errors identify the context of nonfinite values. Each output is written to a temporary file beside its intended destination. ZRust writes and synchronizes its contents before moving the completed file into place. Guide output is completed before gene output. Each file has this protection separately. The two files are not updated as one transaction, and the tests do not establish a guarantee of recovery after a system crash. Evaluation checked successful writes, ordinary failures, repeated output, and the fallback when a worker cannot start.

Measurements used an Apple M4 Pro workstation with 24 GiB RAM and 12 physical cores, including eight performance cores. It used internal APFS storage, macOS 27.0.1, and Rust/Cargo 1.98.0. Five formal repetitions followed an initial smoke run for each configuration. Timing covered process launch, input, calculations, result preparation, writing and synchronizing each file, moving completed files into place, and termination. The Python comparison used ZRust with serial output formatting. It used native elapsed time at 0.01-second resolution. The worker comparison used a high-resolution controller clock that also included launch of the native-time wrapper. Filesystem caches were warm. Unrelated system activity was uncontrolled. No benchmark was rerun for this editorial revision.

The 50,000-guide unpaired test showed a small slowdown relative to a separately timed baseline. This result does not alter the retained overall decision to use eight workers. This comparison must not be combined with the Python comparison.

Data and code availability

ZRust 0.1.0 is an MIT-licensed source implementation. The crate and executable are named zrust. The source code will be available from the ZRust GitHub repository. Validation currently covers the aarch64-apple-darwin target on the tested macOS ARM64 workstation. The published screen count matrices are available through the cited versioned data record [4].

Funding

This work received no specific funding.

Competing interests

The author declares no competing interests.

References

1. Colic M, Wang G, Zimmermann M, et al. Identifying chemogenetic interactions from CRISPR screens with drugZ. Genome Medicine. 2019;11:52. https://doi.org/10.1186/s13073-019-0665-3

2. Benjamini Y, Hochberg Y. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. Journal of the Royal Statistical Society: Series B (Methodological). 1995;57(1):289-300. https://doi.org/10.1111/j.2517-6161.1995.tb02031.x

3. Hart Lab. DrugZ [software]. Commit eb15d34e4dd172965e618d5bb662c053066da799. GitHub source snapshot.

4. Author-released screen count matrices, version 2 [dataset]. https://doi.org/10.6084/m9.figshare.8799215.v2